- A *client unit test*, which starts on the client and ends in the adapter. The adapter's responses are stubbed, meaning that the adapter isn't even making fake server calls. This allows you to unit-test the client completely separate from the server API.
- An *adapter unit test*, which starts in the adapter and ends in the fake server. These tests are the final piece of the chain and allow you to validate the behavior of the adapter separate from any client or the actual server.

Prescription 36 Mediating interaction to an external server through an adapter that is part of your code makes the interaction both easier to test and easier to use.

The Service Integration Test

You'll use the Twitter gem to interface with Twitter.[1] You're also going to need the VCR and Webmock gems in the test environment.[2] [3] Add them to the Gemfile:

```
gem 'twitter'
gem 'vcr', group: :test
gem 'webmock', group: :test
```

You'll also have to reinstall the bundle with `bundle install`.

You need a Twitter API key and a secret key, which you can get from the Twitter Application Management page.[4] In Rails 5, those get placed in the `secrets.yml` file, which typically is not stored in your code repository, though I've put it in the sample code for ease of setup:

external/01/config/secrets.yml

```
shared:
  api_key: 123

development:
  secret_key_base: 1fe3edb870eff077105cff3ed19f0bdf15a5f999ef09ba1043a4916

test:
  secret_key_base: 8baba343e347eadbfcbfa77afb3ba0b13b403fa2dcfb643edf18e44
  twitter_api_key: "FuiCOb9knp5USp6Si1rlAscHa"
  twitter_api_secret: "WYcZZdrPBteO4UCbjqFDfm8SakmeVILgO3tJ7SFADwqLjnPNsJ"

production:
  secret_key_base: <%= ENV["SECRET_KEY_BASE"] %>
```

1. https://github.com/sferik/twitter
2. https://github.com/vcr/vcr
3. https://github.com/bblimke/webmock
4. https://apps.twitter.com